

Міністерство Освіти і Науки України
Національний Університет “Львівська Політехніка”



Кафедра ЕОМ

МНОЖЕННЯ МАТРИЦЬ НА КІЛЬЦЕВІЙ СТРУКТУРІ ЗАСОБАМИ МРІ

Методичні вказівки
до лабораторної роботи з курсу “Паралельні та розподілені обчислення”
для студентів спеціальності F7 “Комп’ютерна інженерія”

Затверджено
на засіданні кафедри
”Електронних обчислювальних машин”
Протокол № _ від 2026 року

Львів – 2026

МЕТА РОБОТИ. Засвоєння основних методів та алгоритмів моделювання паралельних обчислювальних процесів, принципів побудови відповідних структур, набуття початкових практичних навиків проектування таких засобів та ознайомлення з бібліотекою MPI.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Перемноження матриці на матрицю на кільцевій структурі. Множення матриці на вектор і матриці на матрицю є базовими макроопераціями для багатьох задач лінійної алгебри, наприклад ітераційних методів розв'язання систем лінійних рівнянь і т. п. Тому приведені алгоритми тут можна розглядати як фрагменти в алгоритмах цих методів. Розглянемо три алгоритми множення матриці на матрицю. Розмаїтість варіантів алгоритмів виникає із-за розмаїтості обчислювальних систем і розмаїтості розмірів задач. Розглядаються і *різні варіанти завантаження даних* у систему: завантаження даних через один комп'ютер; і завантаження даних безпосередньо кожним комп'ютером з дискової пам'яті. Якщо завантаження даних здійснюється через один комп'ютер, то дані зчитуються цим комп'ютером з дискової пам'яті, розрізаються на частини, які розсилаються іншим комп'ютерам. Але дані можуть бути підготовлені і заздалегідь, тобто заздалегідь розрізані вроздріб і кожна частина записана на диск у виді окремого файлу зі своїм ім'ям; потім кожен комп'ютер безпосередньо зчитує з диска, призначений для нього файл.

Алгоритм - Перемноження матриці на матрицю на кільцевій структурі. Задано дві вихідні матриці A і B . Обчислюється добуток $C = A \times B$, де A – матриця $n_1 \times n_2$, і B – матриця $n_2 \times n_3$. Матриця результатів C має розмір $n_1 \times n_3$. Вихідні матриці попередньо розрізані на смуги, смуги записані на дискову пам'ять окремими файлами зі своїми іменами і доступні всім комп'ютерам. Матриця результатів повертається в нульовий процес.

Оскільки за умовою в комп'ютерах знаходиться по одній смузі матриць, то смуги матриці B (або смуги матриці A) необхідно "прокрутити" по кільцю комп'ютерів повз смуги матриці A (матриці B). На кожному з таких кроків обчислюється тільки частина смуги. Процес і обчислює на j -м кроці добуток i -ї горизонтальної смуги матриці A j -ї вертикальної смуги матриці B , добуток отриманий у підматриці (i, j) матриці C .

Обчислення відбувається в такій послідовності.

1. Кожен комп'ютер зчитує з дискової пам'яті відповідну йому смугу матриці A . Нульова смуга повинна зчитуватися нульовим комп'ютером, перша смуга – першим комп'ютером і т. д., остання смуга – зчитується останнім комп'ютером.

2. Кожен комп'ютер зчитує з дискової пам'яті відповідну йому смугу матриці B . У даному випадку нульова смуга повинна зчитуватися нульовим комп'ютером,

перша смуга – першим комп'ютером і т. д., остання смуга – зчитується останнім комп'ютером.

3. Обчислювальний крок 1. Кожен процес обчислює одну підматрицю добутку. Вертикальні смуги матриці В зсуваються уздовж кільця комп'ютерів.

4. Обчислювальний крок 2. Кожен процес обчислює одну підматрицю добутку. Вертикальні смуги матриці В зсуваються уздовж кільця комп'ютерів. І т. д.

5. Обчислювальний крок p_1-1 . Кожен процес обчислює одну підматрицю добутку. Вертикальні смуги матриці В зсуваються уздовж кільця комп'ютерів.

6. Обчислювальний крок p_1 . Кожен процес обчислює одну підматрицю добутку. Вертикальні смуги матриці В зсуваються уздовж кільця комп'ютерів.

7. Матриця С збирається в нульовому комп'ютері.

Якщо "прокручувати" вертикальні смуги матриці В, то матриця С буде розподілена горизонтальними смугами, а якщо "прокручувати" горизонтальні смуги матриці А, то матриця С буде розподілена вертикальними смугами.

Алгоритм характерний тим, що після кожного кроку обчислень здійснюється обмін даними. Нехай t_u , t_s , і t_p час операцій, відповідно, множення, додавання і пересилання одного числа в сусідній комп'ютер. Тоді сумарний час операцій множень дорівнює:

$$U = (n_1 * n_3 * n_2) * t_u,$$

сумарний час операцій додавань дорівнює:

$$S = (n_1 * n_3 * (n_2 - 1)) * t_s$$

сумарний час операцій пересилань даних по всіх комп'ютерах дорівнює:

$$P = (n_2 * N_3) * (p_1 - 1) * t_p$$

Загальний час обчислень визначимо як:

$$T = (U + S + P) / p_1$$

Відношення часу "обчислень без обмінів" до загального часу обчислень є величина:

$$K = (U + S) / (U + S + P).$$

Якщо час передачі даних великий в порівнянні з часом обчислень, або канали передачі повільні, то ефективність алгоритму буде не висока. Тут не враховується час початкового завантаження і вивантаження даних у пам'ять системи. У смугах матриць може бути різна кількість рядків, а різниця в кількості рядків між смугами - 1. При великих матрицях цим можна знехтувати.

При достатніх ресурсах пам'яті в системі краще використовувати алгоритм, у якому мінімізовані обміни між комп'ютерами в процесі обчислень. Це досягається за рахунок дублювання деяких даних у пам'яті комп'ютерів. Такий підхід використовується, зокрема, при реалізації перемноження матриць на структурі 2D (решітка) або 3D (куб).

Використання технології паралельного програмування Message Passing Interface (MPI) для реалізації паралельного множення матриць. MPI - бібліотека функцій, яка забезпечує взаємодію паралельних процесів за допомогою механізму передачі повідомлень і не має ніяких засобів для розподілення процесів по обчислювальних вузлах і для запуску їх на виконання. MPI не містить механізмів динамічного створення і знищення процесів під час виконання програми.

Для ідентифікації наборів процесів вводиться поняття групи і комунікатора.

Процеси об'єднуються в групи, можуть бути вкладені групи. Усередині групи всі процеси понумеровані. З кожною групою асоційований свій комунікатор. Тому при здійсненні пересилок необхідно вказати ідентифікатор групи, усередині якої проводиться це пересилка.

Процедури MPI:

- ініціалізації та закриття MPI –процесів;
- реалізації комутаційних операцій типу “точка-точка”;
- реалізації колективних операцій;
- для роботи з групами процесів і комунікаторами;
- для роботи з структурами даних;
- формування топології процесів.

До базових функцій MPI відносяться:

- ініціалізація MPI;
- завершення MPI;
- визначення кількості процесів в області зв'язку;
- визначення номеру процесу, який виконується;
- передача повідомлень;
- приймання повідомлень;
- функції відліку часу.

Кожна MPI – функція характеризується способом виконання.

1. Локальна функція – виконується всередині процесу, що її викликав. Її завершення не вимагає комунікацій.

2. Нелокальна функція – для її завершення необхідно виконати MPI – процедуру іншим процесом.

3. Глобальна функція – процедуру повинні виконати всі процеси групи. Невиконання цієї умови може привести до “зависання” задачі.

4. Блокуюча функція – повернення керування з процедури гарантує можливість повторного використання параметрів, які приймали участь у виклику. Ніякої змін в стан процесу, що викликав блокуючий запит до виходу з процедури не може відбуватися.

5. Неблокуюча функція – повернення з процедури відбувається негайно, без очікування завершення операції. Завершення неблокуючих операцій здійснюється спеціальними функціями.

Операції обміну повідомленнями.

Режими обміну:

В загальному випадку є чотири режими обміну: асинхронний (стандартний), синхронний, з буферизацією, по “готовності”.

Обмін типу “точка-точка” – найпростіша форма обміну повідомленнями, в якій приймають участь тільки два процеси: джерело і адресат. Є кілька різновидностей двохточкового обміну:

- синхронний обмін – супроводжується повідомленням про завершення прийому повідомлення;
- асинхронний обмін – таким повідомленням не супроводжується;
- блокуючі прийом/передача – призупиняють виконання процесу на час приймання повідомлення.;
- неблокуючі прийом/передача - виконання процесу продовжується в фоновому режимі, а програма в потрібний момент може запитати підтвердження завершення приймання повідомлення.

Неблокуючий обмін вимагає акуратності при виконанні функцій прийому. Оскільки неблокуючий прийом завершується негайно, для системи неважливо, чи прибуло повідомлення до місця призначення чи ні. Переконатися про це можна за допомогою функції перевірки отримання повідомлення. Звичайно виклик таких функцій розміщується в циклі, який повторюється до тих пір, доки функція перевірки не поверне значення “істина” (перевірка отримання пройшла успішно). Після цього можна викликати функцію прийому повідомлення з буферу повідомлень.

Колективний обмін . В операціях використовуються не два а більше процесів. Різновидностями обміну є:

- широкосмугова передача – передача виконується від одного процесу до всіх;

- обмін з бар'єром – форма синхронізації роботи процесів, коли обмін повідомленнями проходить тільки після того, як до певної процедури звернулась певна кількість процесів;

- операції приведення – вхідними є дані кількох процесів, а результат – одне значення, яке стає доступним всі процесам, які приймали участь в обміні.

Важливою властивістю системи передачі повідомлень є гарантія збереження порядку прийому повідомлень (при відправленні одним процесом іншому кількох повідомлень вони повинні бути прийняті в тій самій послідовності в якій були відправлені). Більшість реалізацій моделі передачі повідомлень забезпечують цю властивість, але не у всіх режимах обміну.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

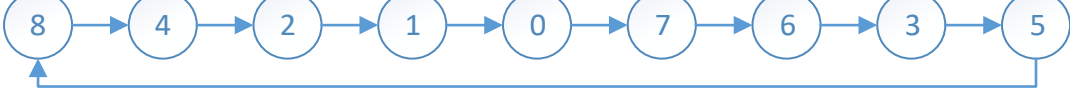
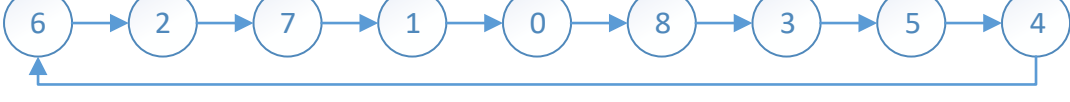
1. Написати програму множення 2 матриць на кільцевій структурі з 8 процесорів, засобами MPI. При цьому слід дотримуватися таких вимог:
 - 1.1. Розмірності матриць та шлях передачі даних (кільце) визначаються варіантом. 0-вий процес розсилає вхідні дані і збирає результат, решта 8 процесів займаються обчисленнями.
 - 1.2. Результат роботи кожного процесу повинен відображатися у тестових файлах з відповідними назвами. Остаточний результат – окремо.
 - 1.3. Передбачити відображення процесу обміну даними(без відображення самих даних) на екрані.
2. Протестувати створену програму і визначити час обчислень. Збільшити розмір матриць у 2, 3, 5 і 10 раз і дослідити час обчислень в залежності від розмірності вхідних даних.
3. Порівняти час обчислень з часом, отриманим у 3-й лабораторній роботі.
4. Зробити висновки про ефективність розпаралелення і визначити кращий варіант розпаралелення – решітка чи кільце.

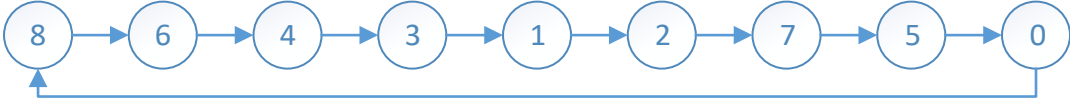
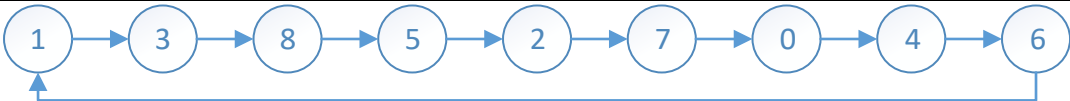
ЗМІСТ ЗВІТУ

1. Тема, мета, аналіз завдання.
2. Розрахунок розмірів частин матриць, що будуть опрацьовуватися окремими процесорами.
3. Результати обчислень в кожному процесорі і остаточний результат множення(скріншоти)
4. Результати дослідження ефективності паралельної програми на структурі кільце у порівнянні з послідовною та на структурі решітка за часом виконання у табличній і графічній формі.
5. Висновки.

ВАРІАНТИ ЗАВДАНЬ

Варіант №	Граф, що відображає конфігурацію зв'язків між процесами		
	N1	N2	N3
1			
	256	541	325
2			
	300	112	214
3			
	113	200	77
4			
	247	111	198
5			
	251	75	65
6			
	232	311	100
7			
	58	145	217
8			
	69	112	124
9			
	196	301	115
10			
	128	89	278

11									
	187			219			120		
12									
	210			211			148		
13									
	222			168			201		
14									
	149			267			215		
15									
	147			214			265		
16									
	217			149			100		
17									
	101			217			220		
18									
	266			149			187		
19									
	197			211			99		
20									
	87			319			213		
21									
	254			117			101		

22									
	107			247			100		
23									
	187			198			99		
24									
	297			143			88		
25									
	183			140			210		
26									
	221			198			97		
27									
	95			159			137		
28									
	181			150			163		
29									
	176			149			152		
30									
	139			284			73		

ПРИКЛАД ВИКОНАННЯ

$n_1 = 60, n_2 = 248, n_3 = 149$

Задано дві матриці A і B . Обчислюється їх добуток $C[n_1][n_3] = A[n_1][n_2] * B[n_2][n_3]$. Матриці A і B попередньо розрізані на 8 смуг (обчислення будуть виконуватись у 8-ми процесах).

Отже, необхідно обчислити $C[60][149] = A[60][248] * B[248][149]$

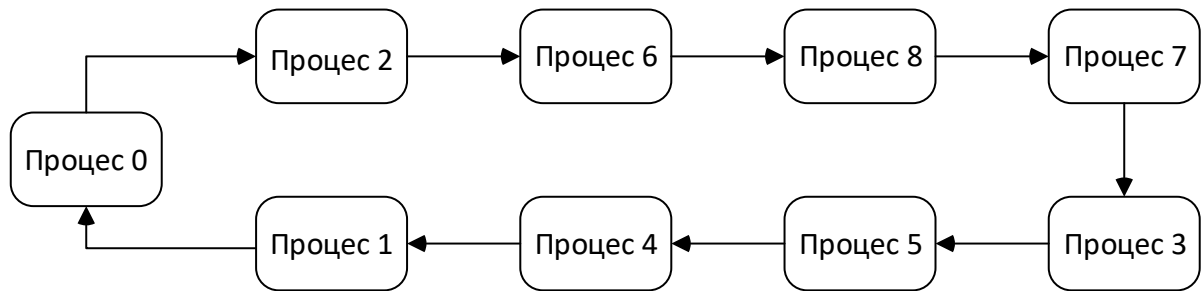


Рис. 1. Граф-схема кільцевого зв'язку між процесами.

Обчислення будуть виконувати 8 процесів, які пов'язані між собою у кільцеву структуру. 0-вий процес буде розсилати вхідні дані до «робочих» процесів і збирати результат в матрицю C . Також в 0-вому процесі буде буфер для пересилки смуг матриці B .

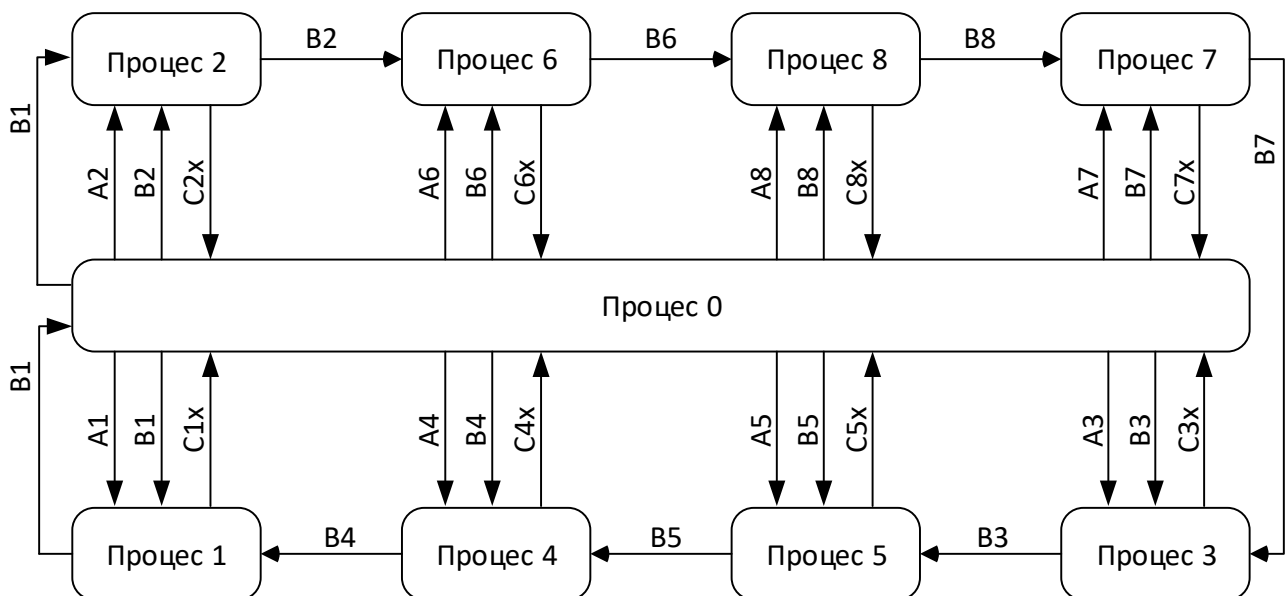


Рис. 2. Структурна схема зв'язків між процесами для структури кільце

Оскільки за умовою в процесах знаходиться по одній смузі матриць, то смуги матриці В (або смуги матриці А) необхідно "прокрутити" по кільцю процесів повз смуги матриці А (матриці В). На кожному з таких кроків обчислюється тільки частина матриці С. Процес і обчислює на j -му кроці добуток i -ї горизонтальної смуги матриці А на j -ту вертикальну смугу матриці В (отримаємо підматрицю C_{ij}).

Обчислення відбувається в такій послідовності.

1. Кожен процес отримує відповідну йому смугу матриці А. Перша смуга повинна потрапити до першого процесу, друга – до другого, і т. д., остання смуга – до останнього процесу.

2. Кожен процес отримує відповідну йому смугу матриці В. Перша смуга повинна потрапити до першого процесу, друга – до другого, і т. д., остання смуга – до останнього процесу.

3. Обчислювальний крок 1. Кожен процес обчислює одну підматрицю С. Вертикальні смуги матриці В зсуваються уздовж кільця процесів.

4. Обчислювальний крок 2. Кожен процес обчислює одну підматрицю С. Вертикальні смуги матриці В зсуваються уздовж кільця процесів.

...

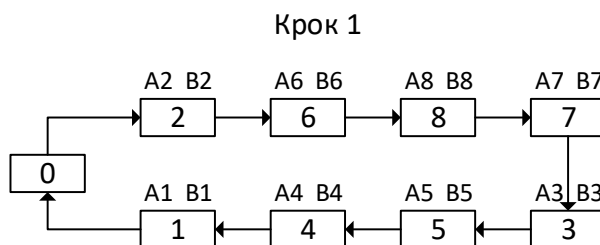
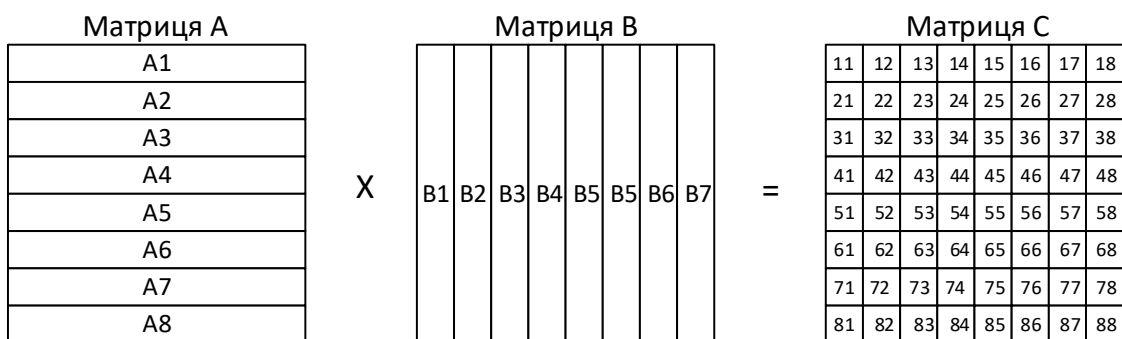
5. Обчислювальний крок 7. Кожен процес обчислює одну підматрицю С. Вертикальні смуги матриці В зсуваються уздовж кільця процесів.

6. Обчислювальний крок 8. Кожен процес обчислює одну підматрицю С. Вертикальні смуги матриці В зсуваються уздовж кільця процесів.

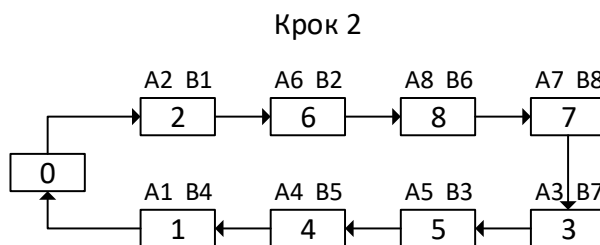
7. Матриця С збирається в нульовому процесі. Усього буде $8 \times 8 = 64$ підматриці С (на кожному з 8-ми обчислювальних кроків кожен з 8-ми процесів обчислює одну підматрицю С).

Алгоритм характерний тим, що після кожного кроку обчислень здійснюється обмін даними між процесами – кожен процес надсилає поточну смугу матриці В сусідньому процесу. У 0-вому процесі передбачено буфер, який приймає смугу матриці В від 1-го процесу. 1-й процес отримує дані від 4-го, 4-й від 5-го, 5-й від 3-го, 3-й від 7-го, 7-й від 8-го, 8-й від 6-го, 6-й від 2-го, а 2-й з буфера 0-го процесу.

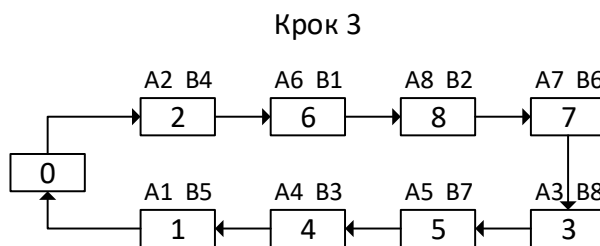
Послідовність обчислень матриці С на кожному з 8-ми кроків показано на наступному рисунку.



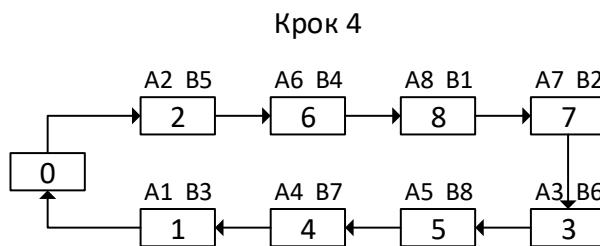
11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88



11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88

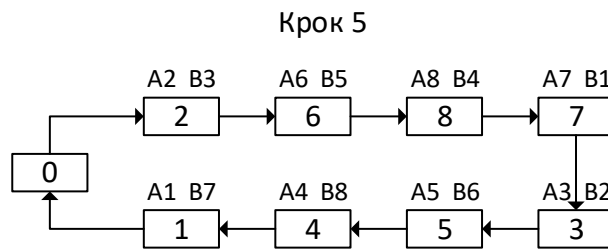


11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88

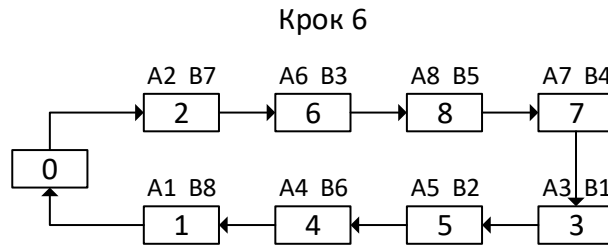


11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88

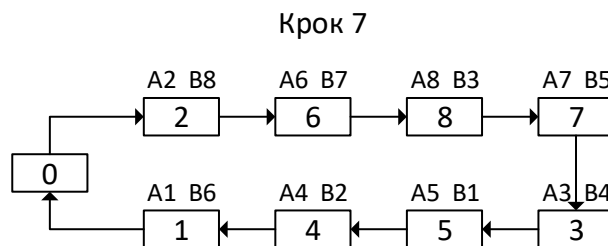
Рис. 3. Послідовність обчислення матриці С.



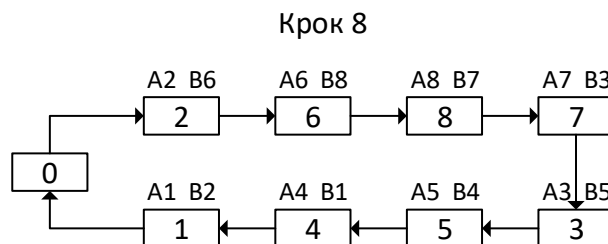
11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88



11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88



11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88



11	12	13	14	15	16	17	18
21	22	23	24	25	26	27	28
31	32	33	34	35	36	37	38
41	42	43	44	45	46	47	48
51	52	53	54	55	56	57	58
61	62	63	64	65	66	67	68
71	72	73	74	75	76	77	78
81	82	83	84	85	86	87	88

Рис. 4. Продовження рис. 3.

Визначення розмірів підматриць (смуг):

Для матриці А поділ відбувається по рядках. Маємо: $60/8=7$ (залишок 4), тобто $60=4*7+4*8$. Таким чином 4 підматриці А будуть мати по 7 рядків і 4 підматриці А будуть мати по 8 рядків. Кількість стовпців – 248.

Для матриці В поділ відбувається по стовпцях. Маємо: $149/8=18$ (залишок 5), тобто $149=3*18+5*19$. Таким чином 3 підматриці В будуть мати по 18 стовпців і 5 підматриць В будуть мати по 19 стовпців. Кількість рядків – 248.

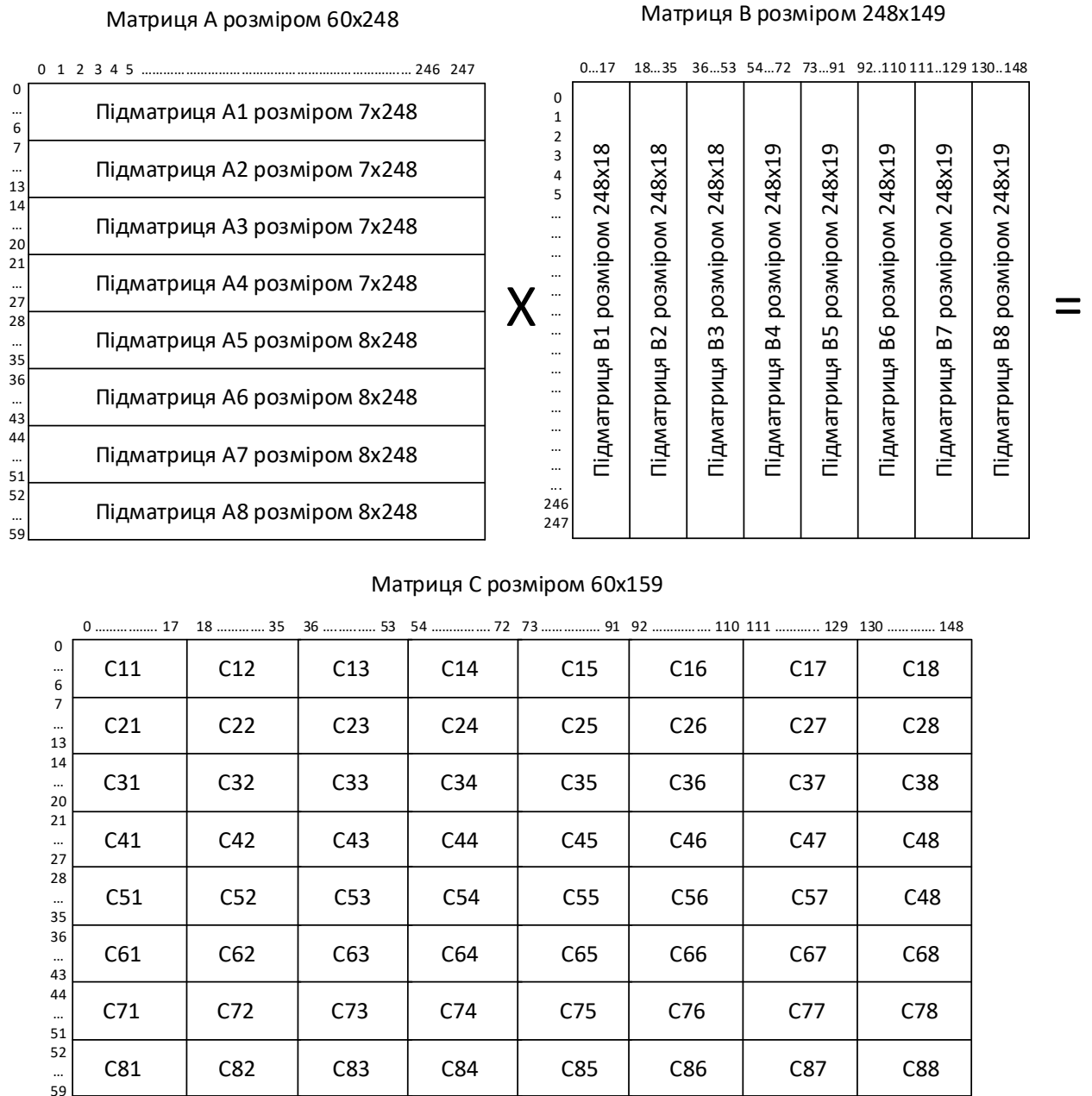


Рис. 5. Схематичне зображення поділу матриць

Визначимо час обчислень для MPI програми на структурі кільце. Збільшимо розмір матриць у 2, 3, 5, 7 і 10 раз і дослідимо час обчислень в залежності від розмірності вхідних даних. Час виконання послідовної програми і MPI програми на структурі решітка візьмемо з 3-ї лабораторної роботи.

Таблиця 1.

Час виконання послідовної, MPI програм на структурі решітка і структурі кільце

Розмір матриць, раз	Час виконання, мс		
	Однопотокова програма	MPI програма на структурі решітка	MPI програма на структурі кільце
1	10,6879	5,8603	15,0340
2	95,7837	39,0821	56,7515
3	351,0731	145,8828	168,8274
5	3753,1181	871,5374	616,0642
7	10958,4770	2694,8283	2388,2424
10	35602,9303	8598,3838	7850,30710

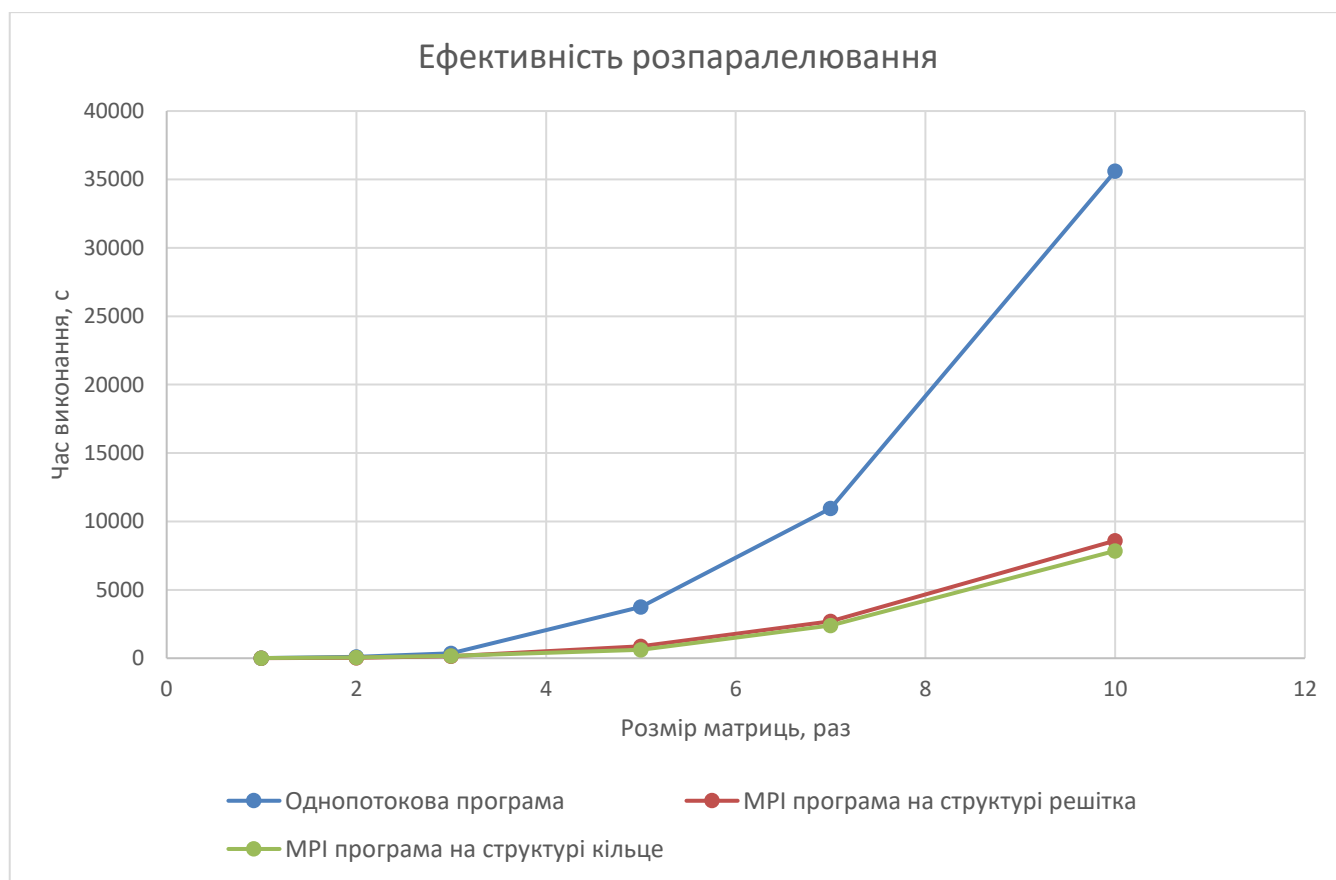


Рис. 6. Графік ефективності розпаралелювання

Висновки. Під час виконання лабораторної роботи, розроблено паралельний алгоритм перемноження матриці на матрицю на кільцевій структурі та його програмну реалізацію. Здійснено тестування програми і порівняння результатів послідовної та паралельної реалізації множення матриць показало ефективність останньої. Ефективність паралельної програми зростає з розміром матриць.

ДОДАТОК А - Результати виконання MPI-програми:

Output the results to the screen and to the file? 1 - yes, else - no : 1

I am 0 process. Send SubMatrix A1 to 1 process
I am 1 process. Received SubMatrix A1 from 0 process
I am 1 process. Received SubMatrix B1 from 0 process
I am 0 process. Send SubMatrix B1 to 1 process
I am 0 process. Send SubMatrix A2 to 2 process
I am 2 process. Received SubMatrix A2 from 0 process
I am 0 process. Send SubMatrix B2 to 2 process
I am 2 process. Received SubMatrix B2 from 0 process
I am 0 process. Send SubMatrix A3 to 3 process
I am 3 process. Received SubMatrix A3 from 0 process
I am 1 process. Multiplied SubMatrix A1 by B1 = SubMatrix C11
I am 3 process. Received SubMatrix B3 from 0 process
I am 0 process. Send SubMatrix B3 to 3 process
I am 0 process. Send SubMatrix A4 to 4 process
I am 4 process. Received SubMatrix A4 from 0 process
I am 2 process. Multiplied SubMatrix A2 by B2 = SubMatrix C22
I am 0 process. Send SubMatrix B4 to 4 process
I am 4 process. Received SubMatrix B4 from 0 process
I am 0 process. Send SubMatrix A5 to 5 process
I am 5 process. Received SubMatrix A5 from 0 process
I am 3 process. Multiplied SubMatrix A3 by B3 = SubMatrix C33
I am 5 process. Received SubMatrix B5 from 0 process
I am 0 process. Send SubMatrix B5 to 5 process
I am 0 process. Send SubMatrix A6 to 6 process
I am 6 process. Received SubMatrix A6 from 0 process
I am 4 process. Multiplied SubMatrix A4 by B4 = SubMatrix C44
I am 6 process. Received SubMatrix B6 from 0 process
I am 0 process. Send SubMatrix B6 to 6 process
I am 0 process. Send SubMatrix A7 to 7 process
I am 7 process. Received SubMatrix A7 from 0 process
I am 5 process. Multiplied SubMatrix A5 by B5 = SubMatrix C55
I am 7 process. Received SubMatrix B7 from 0 process
I am 0 process. Send SubMatrix B7 to 7 process
I am 0 process. Send SubMatrix A8 to 8 process
I am 8 process. Received SubMatrix A8 from 0 process
I am 6 process. Multiplied SubMatrix A6 by B6 = SubMatrix C66
I am 8 process. Received SubMatrix B8 from 0 process
I am 0 process. Send SubMatrix B8 to 8 process
I am 1 process. Send SubMatrix C11 to 0 process
I am 0 process. Received SubMatrix C11 from 1 process
I am 1 process. Send SubMatrix B1 to 0 process
I am 0 process. Received SubMatrix C22 from 2 process
I am 2 process. Send SubMatrix C22 to 0 process
I am 0 process. Received SubMatrix C33 from 3 process
I am 3 process. Send SubMatrix C33 to 0 process
I am 3 process. Send SubMatrix B3 to 5 process
I am 2 process. Send SubMatrix B2 to 6 process

I am 0 process. Received SubMatrix C44 from 4 process
I am 4 process. Send SubMatrix C44 to 0 process
I am 4 process. Send SubMatrix B4 to 1 process
I am 7 process. Multiplied SubMatrix A7 by B7 = SubMatrix C77
I am 0 process. Received SubMatrix C55 from 5 process
I am 5 process. Send SubMatrix C55 to 0 process
I am 5 process. Send SubMatrix B5 to 4 process
I am 6 process. Send SubMatrix C66 to 0 process
I am 0 process. Received SubMatrix C66 from 6 process
I am 6 process. Send SubMatrix B6 to 8 process
I am 0 process. Received SubMatrix C77 from 7 process
I am 7 process. Send SubMatrix C77 to 0 process
I am 7 process. Send SubMatrix B7 to 3 process
I am 4 process. Received SubMatrix B5 from 5 process
I am 5 process. Received SubMatrix B3 from 3 process
I am 1 process. Received SubMatrix B4 from 4 process
I am 8 process. Multiplied SubMatrix A8 by B8 = SubMatrix C88
I am 3 process. Received SubMatrix B7 from 7 process
I am 5 process. Multiplied SubMatrix A5 by B3 = SubMatrix C53
I am 5 process. Sent SubMatrix C53 to 0 process
I am 5 process. Send SubMatrix B3 to 4 process
I am 1 process. Multiplied SubMatrix A1 by B4 = SubMatrix C14
I am 1 process. Sent SubMatrix C14 to 0 process
I am 1 process. Send SubMatrix B4 to 0 process
I am 4 process. Multiplied SubMatrix A4 by B5 = SubMatrix C45
I am 4 process. Sent SubMatrix C45 to 0 process
I am 4 process. Send SubMatrix B5 to 1 process
I am 3 process. Multiplied SubMatrix A3 by B7 = SubMatrix C37
I am 3 process. Sent SubMatrix C37 to 0 process
I am 3 process. Send SubMatrix B7 to 5 process
I am 5 process. Received SubMatrix B7 from 3 process
I am 4 process. Received SubMatrix B3 from 5 process
I am 0 process. Received SubMatrix C88 from 8 process
I am 8 process. Send SubMatrix C88 to 0 process
I am 0 process. Received SubMatrix B1 from 1 process
I am 0 process. Send SubMatrix B1 to 2 process
I am 8 process. Send SubMatrix B8 to 7 process
I am 1 process. Received SubMatrix B5 from 4 process
I am 4 process. Multiplied SubMatrix A4 by B3 = SubMatrix C43
I am 4 process. Sent SubMatrix C43 to 0 process
I am 4 process. Send SubMatrix B3 to 1 process
I am 6 process. Received SubMatrix B2 from 2 process
I am 5 process. Multiplied SubMatrix A5 by B7 = SubMatrix C57
I am 5 process. Sent SubMatrix C57 to 0 process
I am 5 process. Send SubMatrix B7 to 4 process
I am 1 process. Multiplied SubMatrix A1 by B5 = SubMatrix C15
I am 0 process. Received SubMatrix C14 from 1 process
I am 1 process. Sent SubMatrix C15 to 0 process
I am 2 process. Received SubMatrix B1 from 0 process
I am 1 process. Send SubMatrix B5 to 0 process

I am 6 process. Multiplied SubMatrix A6 by B2 = SubMatrix C62
I am 6 process. Sent SubMatrix C62 to 0 process
I am 6 process. Send SubMatrix B2 to 8 process
I am 1 process. Received SubMatrix B3 from 4 process
I am 4 process. Received SubMatrix B7 from 5 process
I am 2 process. Multiplied SubMatrix A2 by B1 = SubMatrix C21
I am 2 process. Sent SubMatrix C21 to 0 process
I am 2 process. Send SubMatrix B1 to 6 process
I am 0 process. Received SubMatrix C21 from 2 process
I am 0 process. Received SubMatrix C37 from 3 process
I am 0 process. Received SubMatrix C45 from 4 process
I am 0 process. Received SubMatrix C53 from 5 process
I am 0 process. Received SubMatrix C62 from 6 process
I am 4 process. Multiplied SubMatrix A4 by B7 = SubMatrix C47
I am 4 process. Sent SubMatrix C47 to 0 process
I am 4 process. Send SubMatrix B7 to 1 process
I am 1 process. Multiplied SubMatrix A1 by B3 = SubMatrix C13
I am 6 process. Received SubMatrix B1 from 2 process
I am 1 process. Sent SubMatrix C13 to 0 process
I am 1 process. Send SubMatrix B3 to 0 process
I am 1 process. Received SubMatrix B7 from 4 process
I am 8 process. Received SubMatrix B6 from 6 process
I am 1 process. Multiplied SubMatrix A1 by B7 = SubMatrix C17
I am 1 process. Sent SubMatrix C17 to 0 process
I am 1 process. Send SubMatrix B7 to 0 process
I am 6 process. Multiplied SubMatrix A6 by B1 = SubMatrix C61
I am 6 process. Sent SubMatrix C61 to 0 process
I am 6 process. Send SubMatrix B1 to 8 process
I am 7 process. Received SubMatrix B8 from 8 process
I am 8 process. Multiplied SubMatrix A8 by B6 = SubMatrix C86
I am 8 process. Sent SubMatrix C86 to 0 process
I am 8 process. Send SubMatrix B6 to 7 process
I am 7 process. Multiplied SubMatrix A7 by B8 = SubMatrix C78
I am 7 process. Sent SubMatrix C78 to 0 process
I am 7 process. Send SubMatrix B8 to 3 process
I am 0 process. Received SubMatrix C78 from 7 process
I am 3 process. Received SubMatrix B8 from 7 process
I am 0 process. Received SubMatrix C86 from 8 process
I am 0 process. Received SubMatrix B4 from 1 process
I am 0 process. Send SubMatrix B4 to 2 process
I am 7 process. Received SubMatrix B6 from 8 process
I am 8 process. Received SubMatrix B2 from 6 process
I am 3 process. Multiplied SubMatrix A3 by B8 = SubMatrix C38
I am 3 process. Sent SubMatrix C38 to 0 process
I am 3 process. Send SubMatrix B8 to 5 process
I am 5 process. Received SubMatrix B8 from 3 process
I am 7 process. Multiplied SubMatrix A7 by B6 = SubMatrix C76
I am 7 process. Sent SubMatrix C76 to 0 process
I am 7 process. Send SubMatrix B6 to 3 process
I am 3 process. Received SubMatrix B6 from 7 process

I am 8 process. Multiplied SubMatrix A8 by B2 = SubMatrix C82
I am 8 process. Sent SubMatrix C82 to 0 process
I am 8 process. Send SubMatrix B2 to 7 process
I am 5 process. Multiplied SubMatrix A5 by B8 = SubMatrix C58
I am 5 process. Sent SubMatrix C58 to 0 process
I am 5 process. Send SubMatrix B8 to 4 process
I am 4 process. Received SubMatrix B8 from 5 process
I am 3 process. Multiplied SubMatrix A3 by B6 = SubMatrix C36
I am 3 process. Sent SubMatrix C36 to 0 process
I am 3 process. Send SubMatrix B6 to 5 process
I am 5 process. Received SubMatrix B6 from 3 process
I am 4 process. Multiplied SubMatrix A4 by B8 = SubMatrix C48
I am 4 process. Sent SubMatrix C48 to 0 process
I am 4 process. Send SubMatrix B8 to 1 process
I am 1 process. Received SubMatrix B8 from 4 process
I am 5 process. Multiplied SubMatrix A5 by B6 = SubMatrix C56
I am 5 process. Sent SubMatrix C56 to 0 process
I am 5 process. Send SubMatrix B6 to 4 process
I am 4 process. Received SubMatrix B6 from 5 process
I am 1 process. Multiplied SubMatrix A1 by B8 = SubMatrix C18
I am 1 process. Sent SubMatrix C18 to 0 process
I am 1 process. Send SubMatrix B8 to 0 process
I am 7 process. Received SubMatrix B2 from 8 process
I am 8 process. Received SubMatrix B1 from 6 process
I am 4 process. Multiplied SubMatrix A4 by B6 = SubMatrix C46
I am 4 process. Sent SubMatrix C46 to 0 process
I am 4 process. Send SubMatrix B6 to 1 process
I am 1 process. Received SubMatrix B6 from 4 process
I am 7 process. Multiplied SubMatrix A7 by B2 = SubMatrix C72
I am 7 process. Sent SubMatrix C72 to 0 process
I am 7 process. Send SubMatrix B2 to 3 process
I am 3 process. Received SubMatrix B2 from 7 process
I am 1 process. Multiplied SubMatrix A1 by B6 = SubMatrix C16
I am 1 process. Sent SubMatrix C16 to 0 process
I am 1 process. Send SubMatrix B6 to 0 process
I am 0 process. Received SubMatrix C15 from 1 process
I am 2 process. Received SubMatrix B4 from 0 process
I am 8 process. Multiplied SubMatrix A8 by B1 = SubMatrix C81
I am 8 process. Sent SubMatrix C81 to 0 process
I am 8 process. Send SubMatrix B1 to 7 process
I am 7 process. Received SubMatrix B1 from 8 process
I am 3 process. Multiplied SubMatrix A3 by B2 = SubMatrix C32
I am 3 process. Sent SubMatrix C32 to 0 process
I am 3 process. Send SubMatrix B2 to 5 process
I am 5 process. Received SubMatrix B2 from 3 process
I am 7 process. Multiplied SubMatrix A7 by B1 = SubMatrix C71
I am 7 process. Sent SubMatrix C71 to 0 process
I am 7 process. Send SubMatrix B1 to 3 process
I am 3 process. Received SubMatrix B1 from 7 process
I am 2 process. Multiplied SubMatrix A2 by B4 = SubMatrix C24

I am 2 process. Sent SubMatrix C24 to 0 process
I am 2 process. Send SubMatrix B4 to 6 process
I am 0 process. Received SubMatrix C24 from 2 process
I am 0 process. Received SubMatrix C38 from 3 process
I am 0 process. Received SubMatrix C43 from 4 process
I am 0 process. Received SubMatrix C57 from 5 process
I am 0 process. Received SubMatrix C61 from 6 process
I am 0 process. Received SubMatrix C76 from 7 process
I am 5 process. Multiplied SubMatrix A5 by B2 = SubMatrix C52
I am 0 process. Received SubMatrix C82 from 8 process
I am 5 process. Sent SubMatrix C52 to 0 process
I am 5 process. Send SubMatrix B2 to 4 process
I am 0 process. Received SubMatrix B5 from 1 process
I am 0 process. Send SubMatrix B5 to 2 process
I am 4 process. Received SubMatrix B2 from 5 process
I am 0 process. Received SubMatrix C13 from 1 process
I am 2 process. Received SubMatrix B5 from 0 process
I am 3 process. Multiplied SubMatrix A3 by B1 = SubMatrix C31
I am 3 process. Sent SubMatrix C31 to 0 process
I am 3 process. Send SubMatrix B1 to 5 process
I am 5 process. Received SubMatrix B1 from 3 process
I am 4 process. Multiplied SubMatrix A4 by B2 = SubMatrix C42
I am 4 process. Sent SubMatrix C42 to 0 process
I am 4 process. Send SubMatrix B2 to 1 process
I am 1 process. Received SubMatrix B2 from 4 process
I am 5 process. Multiplied SubMatrix A5 by B1 = SubMatrix C51
I am 5 process. Sent SubMatrix C51 to 0 process
I am 5 process. Send SubMatrix B1 to 4 process
I am 4 process. Received SubMatrix B1 from 5 process
I am 2 process. Multiplied SubMatrix A2 by B5 = SubMatrix C25
I am 2 process. Sent SubMatrix C25 to 0 process
I am 2 process. Send SubMatrix B5 to 6 process
I am 0 process. Received SubMatrix C25 from 2 process
I am 0 process. Received SubMatrix C36 from 3 process
I am 0 process. Received SubMatrix C47 from 4 process
I am 0 process. Received SubMatrix C58 from 5 process
I am 1 process. Multiplied SubMatrix A1 by B2 = SubMatrix C12
I am 1 process. Sent SubMatrix C12 to 0 process
I am 6 process. Received SubMatrix B4 from 2 process
I am 4 process. Multiplied SubMatrix A4 by B1 = SubMatrix C41
I am 4 process. Sent SubMatrix C41 to 0 process
I am 6 process. Multiplied SubMatrix A6 by B4 = SubMatrix C64
I am 6 process. Sent SubMatrix C64 to 0 process
I am 6 process. Send SubMatrix B4 to 8 process
I am 0 process. Received SubMatrix C64 from 6 process
I am 0 process. Received SubMatrix C72 from 7 process
I am 0 process. Received SubMatrix C81 from 8 process
I am 0 process. Received SubMatrix B3 from 1 process
I am 0 process. Send SubMatrix B3 to 2 process
I am 8 process. Received SubMatrix B4 from 6 process

I am 8 process. Multiplied SubMatrix A8 by B4 = SubMatrix C84
I am 8 process. Sent SubMatrix C84 to 0 process
I am 8 process. Send SubMatrix B4 to 7 process
I am 7 process. Received SubMatrix B4 from 8 process
I am 7 process. Multiplied SubMatrix A7 by B4 = SubMatrix C74
I am 7 process. Sent SubMatrix C74 to 0 process
I am 7 process. Send SubMatrix B4 to 3 process
I am 3 process. Received SubMatrix B4 from 7 process
I am 0 process. Received SubMatrix C17 from 1 process
I am 2 process. Received SubMatrix B3 from 0 process
I am 3 process. Multiplied SubMatrix A3 by B4 = SubMatrix C34
I am 3 process. Sent SubMatrix C34 to 0 process
I am 3 process. Send SubMatrix B4 to 5 process
I am 5 process. Received SubMatrix B4 from 3 process
I am 6 process. Received SubMatrix B5 from 2 process
I am 5 process. Multiplied SubMatrix A5 by B4 = SubMatrix C54
I am 5 process. Sent SubMatrix C54 to 0 process
I am 6 process. Multiplied SubMatrix A6 by B5 = SubMatrix C65
I am 6 process. Sent SubMatrix C65 to 0 process
I am 6 process. Send SubMatrix B5 to 8 process
I am 8 process. Received SubMatrix B5 from 6 process
I am 2 process. Multiplied SubMatrix A2 by B3 = SubMatrix C23
I am 2 process. Sent SubMatrix C23 to 0 process
I am 2 process. Send SubMatrix B3 to 6 process
I am 0 process. Received SubMatrix C23 from 2 process
I am 0 process. Received SubMatrix C32 from 3 process
I am 0 process. Received SubMatrix C48 from 4 process
I am 0 process. Received SubMatrix C56 from 5 process
I am 6 process. Received SubMatrix B3 from 2 process
I am 0 process. Received SubMatrix C65 from 6 process
I am 0 process. Received SubMatrix C71 from 7 process
I am 0 process. Received SubMatrix C84 from 8 process
I am 0 process. Received SubMatrix B7 from 1 process
I am 0 process. Send SubMatrix B7 to 2 process
I am 8 process. Multiplied SubMatrix A8 by B5 = SubMatrix C85
I am 8 process. Sent SubMatrix C85 to 0 process
I am 8 process. Send SubMatrix B5 to 7 process
I am 7 process. Received SubMatrix B5 from 8 process
I am 6 process. Multiplied SubMatrix A6 by B3 = SubMatrix C63
I am 6 process. Sent SubMatrix C63 to 0 process
I am 6 process. Send SubMatrix B3 to 8 process
I am 8 process. Received SubMatrix B3 from 6 process
I am 7 process. Multiplied SubMatrix A7 by B5 = SubMatrix C75
I am 7 process. Sent SubMatrix C75 to 0 process
I am 7 process. Send SubMatrix B5 to 3 process
I am 8 process. Multiplied SubMatrix A8 by B3 = SubMatrix C83
I am 8 process. Sent SubMatrix C83 to 0 process
I am 8 process. Send SubMatrix B3 to 7 process
I am 3 process. Received SubMatrix B5 from 7 process
I am 7 process. Received SubMatrix B3 from 8 process

I am 3 process. Multiplied SubMatrix A3 by B5 = SubMatrix C35
I am 3 process. Sent SubMatrix C35 to 0 process
I am 0 process. Received SubMatrix C18 from 1 process
I am 2 process. Received SubMatrix B7 from 0 process
I am 7 process. Multiplied SubMatrix A7 by B3 = SubMatrix C73
I am 7 process. Sent SubMatrix C73 to 0 process
I am 2 process. Multiplied SubMatrix A2 by B7 = SubMatrix C27
I am 2 process. Sent SubMatrix C27 to 0 process
I am 2 process. Send SubMatrix B7 to 6 process
I am 0 process. Received SubMatrix C27 from 2 process
I am 0 process. Received SubMatrix C31 from 3 process
I am 0 process. Received SubMatrix C46 from 4 process
I am 0 process. Received SubMatrix C52 from 5 process
I am 0 process. Received SubMatrix C63 from 6 process
I am 0 process. Received SubMatrix C74 from 7 process
I am 0 process. Received SubMatrix C85 from 8 process
I am 0 process. Received SubMatrix B8 from 1 process
I am 0 process. Send SubMatrix B8 to 2 process
I am 6 process. Received SubMatrix B7 from 2 process
I am 6 process. Multiplied SubMatrix A6 by B7 = SubMatrix C67
I am 6 process. Sent SubMatrix C67 to 0 process
I am 6 process. Send SubMatrix B7 to 8 process
I am 8 process. Received SubMatrix B7 from 6 process
I am 0 process. Received SubMatrix C16 from 1 process
I am 2 process. Received SubMatrix B8 from 0 process
I am 8 process. Multiplied SubMatrix A8 by B7 = SubMatrix C87
I am 8 process. Sent SubMatrix C87 to 0 process
I am 2 process. Multiplied SubMatrix A2 by B8 = SubMatrix C28
I am 2 process. Sent SubMatrix C28 to 0 process
I am 2 process. Send SubMatrix B8 to 6 process
I am 0 process. Received SubMatrix C28 from 2 process
I am 0 process. Received SubMatrix C34 from 3 process
I am 0 process. Received SubMatrix C42 from 4 process
I am 6 process. Received SubMatrix B8 from 2 process
I am 0 process. Received SubMatrix C51 from 5 process
I am 0 process. Received SubMatrix C67 from 6 process
I am 0 process. Received SubMatrix C75 from 7 process
I am 0 process. Received SubMatrix C83 from 8 process
I am 0 process. Received SubMatrix B6 from 1 process
I am 0 process. Send SubMatrix B6 to 2 process
I am 0 process. Received SubMatrix C12 from 1 process
I am 2 process. Received SubMatrix B6 from 0 process
I am 6 process. Multiplied SubMatrix A6 by B8 = SubMatrix C68
I am 6 process. Sent SubMatrix C68 to 0 process
I am 2 process. Multiplied SubMatrix A2 by B6 = SubMatrix C26
I am 2 process. Sent SubMatrix C26 to 0 process
I am 0 process. Received SubMatrix C26 from 2 process
I am 0 process. Received SubMatrix C35 from 3 process
I am 0 process. Received SubMatrix C41 from 4 process
I am 0 process. Received SubMatrix C54 from 5 process

I am 0 process. Received SubMatrix C68 from 6 process
I am 0 process. Received SubMatrix C73 from 7 process
I am 0 process. Received SubMatrix C87 from 8 process


```

//ДОДАТОК Б – MPI програма
#define _CRT_SECURE_NO_WARNINGS
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <time.h>
#include <windows.h>

#define K 1
#define size1 60*K
#define size2 248*K
#define size3 149*K

double** CreateMatrix(int m, int n)
{
    double** a = (double**)malloc(m * sizeof(double*));
    for (int i = 0; i < m; i++)
        a[i] = (double*)malloc(n * sizeof(double));
    return a;
}

void DeleteMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        free(a[i]);
    free(a);
}

void FillMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            a[i][j] = rand() % 100 + 1;
        }
}

void FillMatrixOne(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            a[i][j] = 1;
        }
}

void PrintMatrixToFile(FILE* f, char* title, double** a, int m, int n)
{
    fprintf(f, title);
    fprintf(f, "\n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            fprintf(f, "%.0f\t", a[i][j]);
        fprintf(f, "\n");
    }
    fprintf(f, "\n");
}

```

```

// c = a * b, a - [m][n], b - [n][k], c - [m][k]
double** MultMatrix(double** c, double** a, double** b, int m, int n, int k)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < k; j++)
        {
            c[i][j] = 0;
            for (int l = 0; l < n; l++)
                c[i][j] = c[i][j] + a[i][l] * b[l][j];
        }
    return c;
}

int main(int argc, char* argv[])
{
    srand((unsigned int)time(NULL));

    MPI_Init(NULL, NULL);

    LARGE_INTEGER frequency, start, end;

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 9)
    {
        printf("This program requires 9 processes !\n");
        fflush(stdout);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int i, j, k;
    int shift;
    int out;

    char SubMatrix[32];
    char fileName[32];

    if (rank == 0)
    {
        double** A = CreateMatrix(size1, size2);
        double** B = CreateMatrix(size2, size3);
        double** C = CreateMatrix(size1, size3);

        sprintf(fileName, "Results_0.txt");
        FILE* file = fopen(fileName, "w");
        if (file == NULL)
        {
            printf("\nError opening file. Check the path and permissions !\n");
            return 1;
        }

        printf("Output the results to the screen and to the file? 1 - yes, else - no :
");
        fflush(stdout);
        scanf("%d", &out);

        for (i = 1; i <= 8; i++)

```

```

        MPI_Send(&out, 1, MPI_INT, i, 0, MPI_COMM_WORLD);

// random input matrix A
FillMatrixOne(A, size1, size2);
PrintMatrixToFile(file, (char*)"Matrix A = ", A, size1, size2);

// random input matrix B
FillMatrixOne(B, size2, size3);
PrintMatrixToFile(file, (char*)"Matrix B = ", B, size2, size3);

QueryPerformanceFrequency(&frequency);
QueryPerformanceCounter(&start);

// розсилання підматриць A i B
int A_rows;
j = 0;
int B_columns;
int B_submatrix;
shift = 0;
for (i = 1; i <= 8; i++)
{
    if (i <= 4)
        A_rows = 7 * K;
    else
        A_rows = 8 * K;

    MPI_Send(&A_rows, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
    for (k = 0; k < A_rows; k++, j++)
        MPI_Send(A[j], size2, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);

    if (out == 1)
    {
        printf("I am 0 process. Send SubMatrix A%d to %d process\n", i,
i);
        fflush(stdout);
    }

    if (i <= 3)
        B_columns = 18 * K;
    else
        B_columns = 19 * K;

    B_submatrix = i;

    MPI_Send(&B_submatrix, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
    MPI_Send(&B_columns, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
    for (k = 0; k < size2; k++)
        MPI_Ssend((B[k] + shift), B_columns, MPI_DOUBLE, i, 0,
MPI_COMM_WORLD);

    if (out == 1)
    {
        printf("I am 0 process. Send SubMatrix B%d to %d process\n",
B_submatrix, i);
        fflush(stdout);
    }

    shift += B_columns;
}
}

```

```

// отримання частини матриці C після першого кроку
j = 0;
shift = 0;
for (i = 1; i <= 8; i++)
{
    MPI_Recv(&A_rows, 1, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    MPI_Recv(&B_columns, 1, MPI_INT, i, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    for (k = 0; k < A_rows; k++, j++)
        MPI_Recv((C[j] + shift), B_columns, MPI_DOUBLE, i, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    if (out == 1)
    {
        printf("I am 0 process. Received SubMatrix C%d%d from %d
process\n", i, i, i);
        fflush(stdout);
    }

    shift += B_columns;
}

// буфер для отримання смуги матриці B від 1 процесу
double** b_temp = CreateMatrix(size2, 19 * K);

for (i = 2; i <= 8; i++)
{
    // отримання смуги матриці B від 1-го процесу
    MPI_Recv(&B_submatrix, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    MPI_Recv(&B_columns, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    for (j = 0; j < size2; j++)
        MPI_Recv(b_temp[j], B_columns, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    if (out == 1)
    {
        printf("I am 0 process. Received SubMatrix B%d from %d process\n",
B_submatrix, 1);
        fflush(stdout);

        printf("I am 0 process. Send SubMatrix B%d to %d process\n",
B_submatrix, 2);
        fflush(stdout);
    }

    // пересилання смуги матриці B 2-му процесу
    MPI_Send(&B_submatrix, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
    MPI_Send(&B_columns, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
    for (j = 0; j < size2; j++)
        MPI_Send(b_temp[j], B_columns, MPI_DOUBLE, 2, 0, MPI_COMM_WORLD);

    // отримання частин матриці C після i-го кроку
    j = 0;
    for (int ii = 1; ii <= 8; ii++)
    {
        MPI_Recv(&A_rows, 1, MPI_INT, ii, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

```

```

MPI_Status ignore;
MPI_Recv(&B_columns, 1, MPI_INT, ii, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
MPI_Recv(&B_submatrix, 1, MPI_INT, ii, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

if (B_submatrix <= 3)
    shift = (B_submatrix - 1) * B_columns;
else
    shift = 54 * K + (B_submatrix - 4) * B_columns;

for (k = 0; k < A_rows; k++, j++)
    MPI_Recv((C[j] + shift), B_columns, MPI_DOUBLE, ii, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

if (out == 1)
{
    printf("I am 0 process. Received SubMatrix C%d%d from %d
process\n", ii, B_submatrix, ii);
    fflush(stdout);
}
}

QueryPerformanceCounter(&end);
double elapsedTime = static_cast<double>(end.QuadPart - start.QuadPart) /
frequency.QuadPart * 1000;
printf("\nElapsed Time: %f milliseconds\n\n", elapsedTime);

PrintMatrixToFile(file, (char*)"Matrix C = ", C, size1, size3);

DeleteMatrix(A, size1, size2);
DeleteMatrix(B, size2, size3);
DeleteMatrix(C, size1, size3);
DeleteMatrix(b_temp, size2, 19 * K);

fclose(file);
}
else
{

    sprintf(fileName, "Results_%d.txt", rank);

    FILE* file = fopen(fileName, "w");
    if (file == NULL)
    {
        printf("Error opening file. Check the path and permissions !\n");
        return 1;
    }

    MPI_Recv(&out, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    // отримання поточним процесом смуги матриці A
    int A_rows;
    MPI_Recv(&A_rows, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    double** a = CreateMatrix(A_rows, size2);

    for (i = 0; i < A_rows; i++)
        MPI_Recv(a[i], size2, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

```

```

    if (out == 1)
    {
        printf("I am %d process. Received SubMatrix A%d from %d process\n", rank,
rank, 0);
        fflush(stdout);

        sprintf(SubMatrix, "SubMatrix A%d = ", rank);
        PrintMatrixToFile(file, SubMatrix, a, A_rows, size2);
    }

    // отримання поточним процесом смуги матриці B
    double** b = CreateMatrix(size2, 19 * K);

    int B_columns, B_submatrix;
    MPI_Recv(&B_submatrix, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    MPI_Recv(&B_columns, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    for (i = 0; i < size2; i++)
        MPI_Recv(b[i], B_columns, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    if (out == 1)
    {
        printf("I am %d process. Received SubMatrix B%d from %d process\n", rank,
B_submatrix, 0);
        fflush(stdout);

        sprintf(SubMatrix, "SubMatrix B%d = ", B_submatrix);
        PrintMatrixToFile(file, SubMatrix, b, size2, B_columns);
    }

    double** c = CreateMatrix(A_rows, 19 * K);

    // множення смуги матриці A на смугу матриці B
    MultMatrix(c, a, b, A_rows, size2, B_columns);

    if (out == 1)
    {
        printf("I am %d process. Multiplied SubMatrix A%d by B%d = SubMatrix
C%d%d \n", rank, rank, B_submatrix, rank, B_submatrix);
        fflush(stdout);
    }

    // надсилання частини матриці C після першого кроку
    MPI_Send(&A_rows, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    MPI_Send(&B_columns, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    for (i = 0; i < A_rows; i++)
        MPI_Ssend(c[i], B_columns, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

    if (out == 1)
    {
        printf("I am %d process. Send SubMatrix C%d%d to %d process\n", rank,
rank, B_submatrix, 0);
        fflush(stdout);

        sprintf(SubMatrix, "SubMatrix C%d%d = A%d x B%d ", rank, B_submatrix,
rank, B_submatrix);
        PrintMatrixToFile(file, SubMatrix, c, A_rows, B_columns);
    }
}

```

```

int from, to;
switch (rank)
{
case 1: from = 4; to = 0; break;
case 2: from = 0; to = 6; break;
case 3: from = 7; to = 5; break;
case 4: from = 5; to = 1; break;
case 5: from = 3; to = 4; break;
case 6: from = 2; to = 8; break;
case 7: from = 8; to = 3; break;
case 8: from = 6; to = 7; break;
}

for (i = 2; i <= 8; i++)
{
    // пересилання смуги матриці B від поточного процесу до процесу з рангом
to
    MPI_Send(&B_submatrix, 1, MPI_INT, to, 0, MPI_COMM_WORLD);
    MPI_Send(&B_columns, 1, MPI_INT, to, 0, MPI_COMM_WORLD);
    for (j = 0; j < size2; j++)
        MPI_Send(b[j], B_columns, MPI_DOUBLE, to, 0, MPI_COMM_WORLD);

    if (out == 1)
    {
        printf("I am %d process. Send SubMatrix B%d to %d process\n",
rank, B_submatrix, to);
        fflush(stdout);
    }

    // отримання смуги матриці B від процесу з рангом from поточним процесом
    MPI_Recv(&B_submatrix, 1, MPI_INT, from, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    MPI_Recv(&B_columns, 1, MPI_INT, from, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    for (j = 0; j < size2; j++)
        MPI_Recv(b[j], B_columns, MPI_DOUBLE, from, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    if (out == 1)
    {
        printf("I am %d process. Received SubMatrix B%d from %d
process\n", rank, B_submatrix, from);
        fflush(stdout);

        sprintf(SubMatrix, "SubMatrix B%d = ", B_submatrix);
        PrintMatrixToFile(file, SubMatrix, b, size2, B_columns);
    }

    // множення смуги матриці A на смугу матриці B
    MultMatrix(c, a, b, A_rows, size2, B_columns);

    if (out == 1)
    {
        printf("I am %d process. Multiplied SubMatrix A%d by B%d =
SubMatrix C%d%d \n", rank, rank, B_submatrix, rank, B_submatrix);
        fflush(stdout);

        sprintf(SubMatrix, "SubMatrix C%d%d = A%d x B%d ", rank,
B_submatrix, rank, B_submatrix);
        PrintMatrixToFile(file, SubMatrix, c, A_rows, B_columns);
    }
}

```

```

        // надсилання частини матриці C після i-го кроку
        MPI_Send(&A_rows, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Send(&B_columns, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Send(&B_submatrix, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        for (j = 0; j < A_rows; j++)
            MPI_Send(c[j], B_columns, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

        if (out == 1)
        {
            printf("I am %d process. Sent SubMatrix C%d%d to %d process\n",
rank, rank, B_submatrix, 0);
            fflush(stdout);
        }

        DeleteMatrix(a, A_rows, size2);
        DeleteMatrix(b, size2, 19 * K);
        DeleteMatrix(c, A_rows, 19 * K);

        fclose(file);
    }

    MPI_Finalize();
    return 0;
}

```